

Lucas Bueno Cesario

**Polyglot Import CSV: Uma solução para
importação de documentos CSV para múltiplos
bancos de dados**

Florianópolis

2026/1

Lucas Bueno Cesario

Polyglot Import CSV: Uma solução para importação de documentos CSV para múltiplos bancos de dados

Trabalho de conclusão de curso apresentado como parte dos requisitos para obtenção do título de Bacharel, do Curso de Graduação em Ciência da Computação na Universidade Federal de Santa Catarina.

Universidade Federal de Santa Catarina

Orientador: Prof^o Ronaldo dos Santos Mello, Dr.

Florianópolis

2026/1

Lucas Bueno Cesario

Polyglot Import CSV: Uma solução para importação de documentos CSV para múltiplos bancos de dados

Trabalho de conclusão de curso apresentado como parte dos requisitos para obtenção do título de Bacharel, do Curso de Graduação em Ciência da Computação na Universidade Federal de Santa Catarina.

Banca Examinadora:

Prof^o Ronaldo dos Santos Mello, Dr.

Orientador

Universidade Federal de Santa Catarina

Prof^a Carina Friedrich Dorneles, Dra.

Universidade Federal de Santa Catarina

Prof^o Jônata Tyska Carvalho, Dr.

Universidade Federal de Santa Catarina

Florianópolis

2026/1

Resumo

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em aplicativos. A ferramenta “Polyglot Import CSV” destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados, superando desafios da persistência poliglota. Por meio de configurações simples em arquivos JSON, a ferramenta oferece suporte para PostgreSQL, Redis, Cassandra, MongoDB e Neo4j, acelerando o processo e aumentando a flexibilidade para os desenvolvedores. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados, consolidando-se como uma solução versátil e eficiente.

Palavras-chave: Persistência poliglota; Ferramenta “Polyglot Import CSV”; Importação de dados CSV; JSON; SQL; NoSQL; Software Livre.

Lista de ilustrações

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce. . .	10
Figura 2 – Persistência poliglota em ambiente de nuvem (Microsoft Azure).	11
Figura 3 – Diferentes tipos de modelos de dados NoSQL.	11
Figura 4 – Algoritmo de execução da importação Polyglot Import CSV.	27

Lista de abreviaturas e siglas

API	<i>Application Programming Interface</i>
BSON	<i>Binary JSON</i>
CLI	<i>Command Line Interface</i>
CSV	<i>Comma-Separated Values</i>
DDL	<i>Data Definition Language</i>
JSON	<i>JavaScript Object Notation</i>
NoSQL	<i>Not Only SQL</i> ou <i>Not SQL</i>
SGBD	Sistema de Gerenciamento de Banco de Dados
SGPD	Sistema de Gerenciamento Poliglota de Dados
TCC	Trabalho de Conclusão de Curso
UFSC	Universidade Federal de Santa Catarina

Sumário

	Sumário	6
0.1	1 INTRODUÇÃO	7
0.1.1	1.1 Visão Geral	7
0.1.2	1.2 Objetivos	8
0.1.2.1	1.2.1 Objetivo Geral	8
0.1.2.2	1.2.2 Objetivos Específicos	8
0.1.3	1.3 Metodologia	8
0.1.4	1.4 Estrutura do Trabalho	9
0.2	2 FUNDAMENTAÇÃO TEÓRICA	9
0.2.1	2.1 Persistência Poliglota	9
0.2.2	2.2 Bancos de Dados NoSQL	11
0.2.2.1	2.2.1 Chave-valor	11
0.2.2.2	2.2.2 Colunar	12
0.2.2.3	2.2.3 Documento	13
0.2.2.4	2.2.4 Grafo	13
0.2.3	2.3 Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota	13
0.3	3 TRABALHOS RELACIONADOS	14
0.3.1	3.1 Importação de arquivo CSV para um único SGBD	14
0.3.1.1	3.1.1 PostgreSQL	14
0.3.1.2	3.1.2 Redis	15
0.3.1.3	3.1.3 MongoDB	16
0.3.1.4	3.1.4 Cassandra	16
0.3.1.5	3.1.5 Neo4j	17
0.3.2	3.2 Importação de arquivo CSV para SGBD multimodelo	17
0.3.3	3.3 Considerações sobre os trabalhos relacionados	18
0.4	4 A PROPOSTA DA FERRAMENTA PolyglotImportCSV	19
0.4.1	4.1 Visão geral	19
0.4.2	4.2 Formato de configuração (JSON Schema)	19
0.4.2.1	4.2.1 Estrutura raiz	20
0.4.2.2	4.2.2 Mapeamento de colunas (columnSpec)	20
0.4.2.3	4.2.3 Entidade (entity)	21
0.4.2.4	4.2.4 Blocos por backend	23
0.4.2.5	4.2.5 Validação estática e cruzada	24
0.4.3	4.3 Algoritmo de execução da importação	24
0.4.3.1	4.3.1 Fases do algoritmo	24

0.4.3.2 4.3.2 Pseudocódigo 25

0.4.3.3 4.3.3 Diagrama do fluxo 26

0.4.3.4 4.3.4 Exemplo: uma linha action=stock no e-commerce 28

0.4.4 4.4 Validação e execução 28

0.4.4.1 4.4.1 Evidência de execução (cenário e-commerce) 29

0.5 5 ATIVIDADES FUTURAS 29

0.6 CONSIDERAÇÕES FINAIS 29

REFERÊNCIAS 31

0.1 1 INTRODUÇÃO

0.1.1 1.1 Visão Geral

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL (Sadalage; Fowler, 2013).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados (Chillón et al., 2023), comparação de desempenho de bancos de dados multi-modelos contra abordagens de persistência poliglota (Ye et al., 2023), revisões sistemáticas sobre modelagem poliglota de dados (Silva; Bornia; Mello, 2024), tutoriais sobre o estado da arte e desafios abertos (Kiehn et al., 2022), e a adoção da persistência poliglota em Big Data (Khine; Wang, 2019).

Percebe-se a falta de uma ferramenta única de importação de dados que possa realizar a persistência poliglota para diversos SGBDs que tratem modelos de dados diferentes. A proposta deste trabalho é apresentar a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para diferentes sistemas, baseada em uma configuração em JSON que aponta quais colunas do CSV formam cada entidade e relacionamento de cada SGBD.

CSV (Comma-Separated Values) é um arquivo de valores separados por vírgulas, frequentemente visualizado no Excel ou em alguma outra ferramenta de planilha. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de arquivo para outros sistemas, relatórios amigáveis para humanos e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

0.1.2 1.2 Objetivos

0.1.2.1 1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

0.1.2.2 1.2.2 Objetivos Específicos

- Fazer com que o arquivo de configuração JSON seja bem personalizável, seguindo uma sintaxe específica, porém flexível, além de adicionar opções como filtros e escolha de chave primária;
- Não permitir sintaxes de configuração incorretas; se a sintaxe estiver errada em algum lugar do arquivo de configuração, a importação não deve começar até que o arquivo de configuração esteja corrigido;
- Testar a ferramenta com um conjunto de dados armazenados num arquivo CSV que representa dados de um site de e-commerce.

0.1.3 1.3 Metodologia

Buscando obter conhecimento para realização do trabalho, primeiramente foi feito um estudo básico sobre o assunto, seguindo da implementação da solução, como descrito abaixo:

- Estudo sobre persistência poliglota e seu estado da arte;
- Estudo sobre os paradigmas relacional e não-relacionais que serão abrangidos pela solução, tais como: chave-valor, documento, grafo e colunar;
- Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;
- Implementação da solução “Polyglot Import CSV”;
- Testes da solução “Polyglot Import CSV” implementada.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

As etapas de implementação e testes foram concluídas durante a elaboração deste TCC I, resultando no protótipo funcional descrito no Capítulo 4, validado com o conjunto de dados `ecommerce_join.csv` e o arquivo `import_config.json` do cenário e-commerce.

0.1.4 1.4 Estrutura do Trabalho

Este trabalho está estruturado em cinco capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste TCC. O capítulo 2 revisa fundamentos teóricos. O capítulo 3 apresenta trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* e o capítulo 5 as atividades futuras (TCC II).

0.2 2 FUNDAMENTAÇÃO TEÓRICA

0.2.1 2.1 Persistência Poliglota

Em 2008, Neal Ford, em seu livro *The Productive Programmer* (Ford, 2008), introduziu o conceito de “programação poliglota” para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas.

Uma das consequências notáveis da programação poliglota é a transição para a “persistência poliglota”, usando a mesma analogia: pode-se dizer que múltiplas tecnologias de bancos de dados podem ser utilizados como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados em sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (Fowler, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Especificamente, dados estruturados geralmente englobam formatos como relacional, chave-valor e em grafo. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, bem como dados de bancos colunares (Ye et al., 2023).

A seguir, há um exemplo de aplicação prática da persistência poliglota em um e-commerce:

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Esses dados podem ser armazenados em um banco de dados relacional, como o PostgreSQL.
- **Dados do Catálogo de Produtos:** Esses dados podem ser armazenados em um banco de dados NoSQL baseado em documentos, como o MongoDB.
- **Dados da Sessão do Carrinho de Compras e do Usuário:** Esses dados podem ser armazenados em um banco de dados NoSQL baseado em chave/valor, como o Redis.
- **Dados de Atividade do Usuário:** Estes dados podem ser armazenados em bancos de dados NoSQL colunares como o Apache Cassandra.
- **Dados de Recomendação:** Esses dados de recomendação podem ser armazenados em um banco de dados NoSQL baseado em grafos, como o Neo4j.

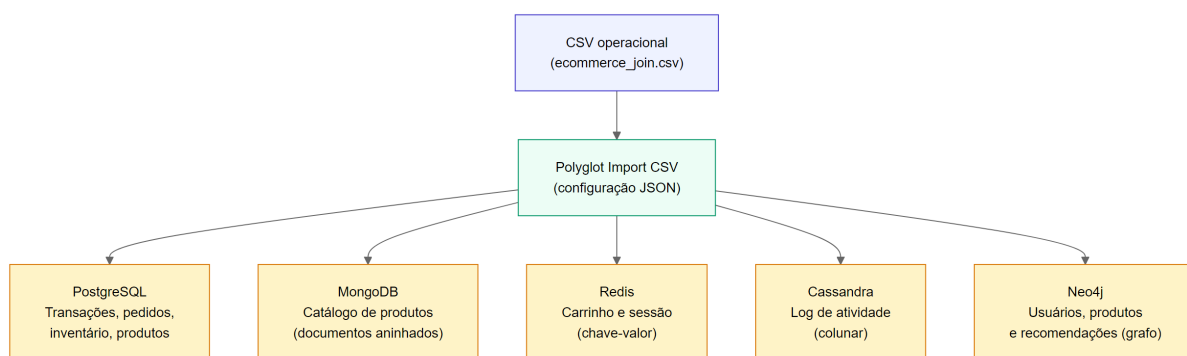


Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.

Fonte: Elaborado pelo autor (2026).

A Figura 2 apresenta um exemplo arquitetural de persistência poliglota em ambiente de nuvem (Microsoft Azure), no qual diferentes categorias de armazenamento atendem a casos de uso distintos (catálogo, carrinho, transações, logs etc.).

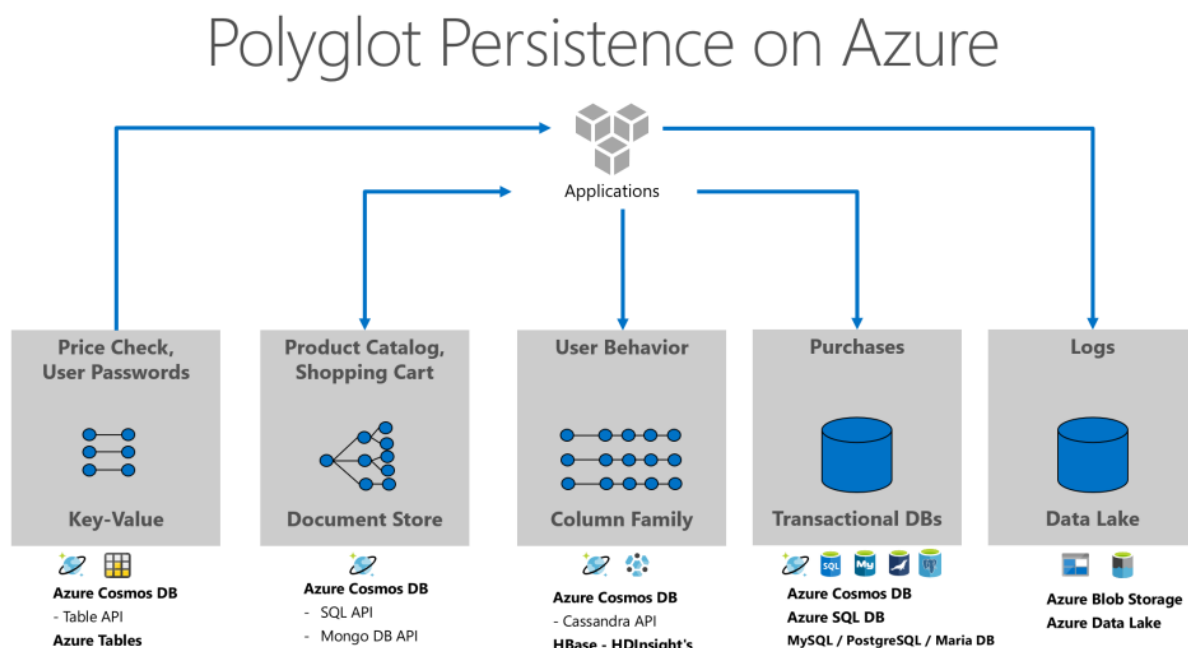


Figura 2 – Persistência poliglota em ambiente de nuvem (Microsoft Azure).

Fonte: Microsoft (s.d.). Adaptado pelo autor (2026).

0.2.2 2.2 Bancos de Dados NoSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de Hecht e Jablonski (Hecht; Jablonski, 2011), que categoriza os bancos de dados NoSQL em quatro tipos principais: chave-valor, colunas, documentos e grafos.

A Figura 3 ilustra esses modelos, de forma respectiva:

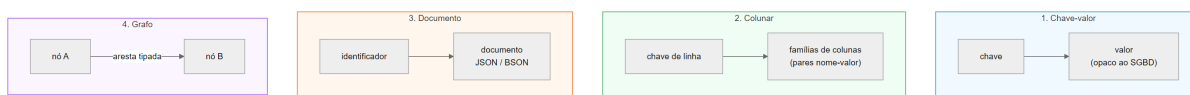


Figura 3 – Diferentes tipos de modelos de dados NoSQL.

Fonte: Elaborado pelo autor (2026).

0.2.2.1 2.2.1 Chave-valor

Os bancos de dados de chave-valor têm um modelo de dados simples baseado em pares de chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco

de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma string, um array ou um objeto, proporcionando um modelo de dados sem esquema.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem relações ou estruturas. Qualquer funcionalidade que requeira relações, estruturas ou ambas deve ser implementada na aplicação cliente que interage com o banco de dados de chave-valor.

Ora, como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves.

Um exemplo de banco de dados chave-valor é o Redis, que mantém os dados tanto na memória como no disco.

0.2.2.2 2.2.2 Colunar

Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, também conhecida como chave primária. Cada linha é composta por um conjunto de famílias de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, cada família de colunas serve ainda como uma chave para uma ou mais colunas que contém, onde cada coluna consiste em um par nome-valor. O Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha por vários nós de servidor usando chaves de partição compostas.

Nos bancos de dados de famílias de colunas, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

Em geral, os bancos de dados de famílias de colunas fornecem capacidades de indexação e consulta mais poderosas do que os bancos de dados de chave-valor porque utilizam famílias de colunas e colunas além das chaves de linha.

Semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente.

0.2.2.3 2.2.3 Documento

Bancos de dados orientados a documento armazenam registros como documentos autocontidos, em geral em JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos da mesma coleção, o que favorece esquemas flexíveis e agregados de leitura (*read-optimized*). O MongoDB é um exemplo popular: coleções de documentos, índices secundários e pipelines de agregação permitem consultas ricas sem exigir junções (*joins*) no servidor como no modelo relacional clássico.

Para o *Polyglot Import CSV*, o modelo documental é adequado a entidades com subestruturas aninhadas (por exemplo, um pedido com subdocumentos **buyer** e **product**), desde que o CSV de origem exponha colunas planas que o mapeamento JSON agrupe em subdocumentos.

0.2.2.4 2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para **MERGE/CREATE** de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador → vendedor com atributos do pedido).

0.2.3 2.3 Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota

SGBDs **multimodelo** integram mais de um paradigma de dados no mesmo motor (por exemplo, documento + chave-valor + grafo), oferecendo em geral uma interface de acesso unificada — como a linguagem AQL do ArangoDB, que combina filtros sobre JSON e travessias em grafos. Essa abordagem reduz o número de produtos operados, mas concentra riscos de *lock-in* e de *tuning* em um único fornecedor (Ye et al., 2023).

A **persistência poliglota**, por outro lado, combina **vários SGBDs heterogêneos**, cada um escolhido pela adequação a um caso de uso (Sadalage; Fowler, 2013). Na literatura recente, essa ideia materializa-se como um **SGPD** (*Sistema de Gerenciamento Poliglota de Dados*): coordenação transparente entre tecnologias distintas, com requisitos de modelagem, expressividade de acesso (DDL/DML, consultas *key-based*, *set-based* e por caminhos em grafos) e capacidade de adaptação quando o *workload* evolui (Blake et al., 2022; Kiehn et al., 2022).

Tan et al. (2017) apresentam uma taxonomia útil para distinguir arquiteturas relacionadas: **BD federado** (fontes homogêneas, interface única), **BD multistore** (fontes heterogêneas, modelo canônico de consulta), **BD polystore** (fontes heterogêneas, múltiplas interfaces de acesso — exemplificadas por sistemas como o BigDAWG) e abordagens **multimodelo** (vários modelos no mesmo motor). A escolha entre essas arquiteturas não é trivial: Roy-Hubara; Shoval; Sturm (2022) propõem critérios para selecionar quais bancos usar em aplicações de persistência poliglota, destacando o trade-off entre especialização por SGBD e simplicidade operacional de um motor multimodelo.

Silva; Bornia; Mello (2024) conduzem revisão sistemática sobre **modelagem poliglota de dados** — como particionar um esquema conceitual em fragmentos lógicos/físicos adequados a tecnologias distintas. Esse trabalho trata da fase de **projeto**; o presente TCC complementa-o na fase **operacional**, abordando a carga inicial de dados a partir de CSV.

O *Polyglot Import CSV* adota explicitamente a linha da persistência poliglota (não a multimodelo): o usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de acesso desejado, alinhado ao cenário e-commerce descrito na seção 2.1 e aos casos de uso discutidos por Kiehn et al. (2022).

0.3 3 TRABALHOS RELACIONADOS

0.3.1 3.1 Importação de arquivo CSV para um único SGBD

0.3.1.1 3.1.1 PostgreSQL

Primeiramente, você especifica a tabela com os nomes das colunas após a palavra-chave **COPY**. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV.

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente.

Segundo, você insere o caminho do arquivo CSV após a palavra-chave **FROM**. Como o formato do arquivo é CSV, você precisa especificar **DELIMITER**, bem como as cláusulas **CSV**.

Terceiro, especifique a palavra-chave **HEADER** para indicar que o arquivo CSV contém um cabeçalho. Quando o comando **COPY** importa dados, ele ignora o cabeçalho do arquivo.

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Observe que o arquivo deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter acesso de superusuário para executar com sucesso a instrução `COPY`.

0.3.1.2 3.1.2 Redis

A maneira preferida de importar dados em massa no Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos.

Por exemplo, se precisar gerar um grande conjunto de dados com bilhões de chaves no formato “chaveN -> ValorN”, deve ser um arquivo contendo os seguintes comandos no formato de protocolo Redis:

```
SET Chave0 Valor0
SET Chave1 Valor1
...
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis o mais rápido possível. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando.

Nas versões 2.6 ou posteriores do Redis, a utilidade `redis-cli` suporta um novo modo chamado modo pipe que foi projetado para realizar o carregamento em massa.

Usando o modo pipe, o comando a ser executado parece o seguinte:

```
cat dados.txt | redis-cli --pipe
```

Isso produzirá uma saída semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A utilidade `redis-cli` também garantirá redirecionar apenas os erros recebidos da instância Redis para a saída padrão.

0.3.1.3 3.1.3 MongoDB

Como a ferramenta **mongoimport** é fornecida oficialmente, o processo de importar dados em formato CSV é muito simples. É uma ferramenta poderosa e fácil de usar. Segue abaixo a sua sintaxe:

```
mongoimport --db nome_do_db --collection nome_da_colecao \  
--type csv --headerline --ignoreBlanks \  
--file caminho/do/arquivo.csv
```

- **--db nome_do_db**: define em qual banco de dados importar os dados.
- **--collection nome_da_colecao**: define o nome da nova coleção. Se esse parâmetro for omitido, o nome da coleção será o mesmo que o nome do arquivo CSV.
- **--type csv**: o tipo de arquivo é CSV.
- **--headerline**: o conteúdo da primeira linha do CSV será o nome de cada campo.
- **--ignoreBlanks**: esse parâmetro pode ignorar valores em branco no arquivo.
- **--file**: arquivo CSV a ser importado.

0.3.1.4 3.1.4 Cassandra

Importar dados CSV para o Cassandra usando **sstableloader** envolve vários passos. O **sstableloader** é uma ferramenta de linha de comando fornecida com o Apache Cassandra para carregar grandes volumes de dados eficientemente.

Certifique-se de que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondam ao esquema da tabela no Cassandra.

O **sstableloader** requer que os dados estejam em um formato específico chamado SSTable. Para converter o CSV em SSTables, você pode usar a ferramenta **cqlsh** fornecida com o Cassandra. Execute algo como:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Agora você pode usar o **sstableloader** para carregar os dados. O comando geralmente se parece com isso:

```
sstableloader -d <hostname> -u <username> -pw <password> \  
output_directory/keyspace_name/table_name
```

- **<hostname>**: O endereço do nó Cassandra.
- **<username>**: O nome de usuário, se a autenticação estiver habilitada.

- `<password>`: A senha correspondente.

Este comando moverá os dados do diretório de saída para o Cassandra usando o `sstableloader`.

0.3.1.5 3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos. Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos.

Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
--from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV.

Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
--mode=csv \
--nodes:Rotulo1 nos.csv \
--nodes:Rotulo2 nos2.csv \
--relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

0.3.2 3.2 Importação de arquivo CSV para SGBD multimodelo

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/gráfo) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco backends independentes.

- **mongoimport** / **COPY** / **redis-cli --pipe** / **cqlsh** / **neo4j-admin** (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** (“dbcrossbar”, 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.
- **Hackolade** (Hackolade, 2024) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

0.3.3 3.3 Considerações sobre os trabalhos relacionados

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema (Berlanga et al., 2021)) e sobre **evolução de esquema** em ambientes heterogêneos (Chillón et al., 2023) informa decisões de validação e de representação de entidades no JSON de configuração, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Surveys recentes convergem em desafios abertos da gestão poliglota de dados: migração automatizada entre stores quando o *workload* muda, planejamento de consultas entre sistemas, gestão de esquemas multi-modelo e preservação das propriedades não funcionais de cada SGBD (Glake et al., 2022; Kiehn et al., 2022). Roy-Hubara; Shoval; Sturm (2022) tratam da **seleção de SGBDs** por fragmento de aplicação; Silva; Bornia; Mello (2024) mapeiam o estado da arte em **modelagem poliglota**. Nenhum desses trabalhos implementa, contudo, um utilitário declarativo que receba um CSV operacional único e o materialize simultaneamente em cinco paradigmas distintos com validação estática prévia.

Sistemas **polystore** como o BigDAWG (Duggan et al., 2017) e middlewares de portabilidade em nuvem (Wang et al., 2020) tratam de **consultas federadas** e movimentação de dados em ecossistemas complexos. Tan et al. (2017) classificam essas soluções (federado, multistore, polystore, poliglota) e avaliam transparência, autonomia e heterogeneidade — eixos nos quais importadores nativos (seção 3.1) e ferramentas de migração (seção 3.2) também divergem, mas sem convergir para um pipeline único de ingestão.

Ferramentas como **dbcrossbar** (“dbcrossbar”, 2024) e **Hackolade** (Hackolade, 2024) aproximam-se do domínio de integração de dados, porém com foco em *pipelines* de migração ou modelagem visual, respectivamente — não em regras declarativas sobre entidades aninhadas, filtros por valor (**each**) e validação por JSON Schema antes de qualquer conexão com SGBDs.

Lacuna identificada: a combinação de (i) um CSV largo como fonte operacional, (ii) configuração JSON validada estaticamente, (iii) materialização distinta por paradigma (relacional, chave-valor, documento, colunar, grafo) e (iv) modo *dry-run* para revisão prévia. O *Polyglot Import CSV* ocupa essa lacuna como ferramenta de **bootstrap de persistência poliglota** — carga inicial e reprodutível de dados — enquanto middlewares e polystores assumem dados já residentes ou consultas em tempo de execução. Trabalhos futuros podem integrar a ferramenta a mecanismos de migração adaptativa descritos por Kiehn et al. (2022) e Glake et al. (2022).

0.4 4 A PROPOSTA DA FERRAMENTA PolyglotImportCSV

0.4.1 4.1 Visão geral

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) lê um arquivo CSV e um arquivo de configuração JSON validado por um **JSON Schema** embutido. A CLI executa, em sequência:

1. Carregamento do CSV (como texto, para evitar erros de *parse* em campos com + em timestamps).
2. Inferência de *kinds* de coluna (inteiro, *float*, data/hora, texto) para validar operadores de filtro.
3. Validação cruzada: colunas referenciadas existem no CSV; relacionamentos PostgreSQL e Neo4j referenciam entidades conhecidas; chaves de partição Cassandra estão declaradas quando necessário.
4. Para cada backend configurado: aplicação de filtros (incluindo o operador **each** para particionar por valor distinto), materialização de entidades e importação (ou apenas resumo em `--dry-run`).

Comando principal:

```
python -m polyglotimportcsv caminho/dados.csv \
  --config caminho/import_config.json \
  [--dry-run] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis]
```

0.4.2 4.2 Formato de configuração (JSON Schema)

O arquivo de configuração é um JSON validado estaticamente pelo **JSON Schema** embutido em `src/polyglotimportcsv/schemas/polyglot_import_config.schema.json`

(rascunho 2020-12). A validação ocorre em `config_parser.load_config` antes de qualquer leitura do CSV ou conexão com SGBDs. O exemplo de referência do cenário e-commerce está em `data/ecommerce/import_config.json`, alinhado ao CSV `data/ecommerce/ecommerce`.

0.4.2.1 4.2.1 Estrutura raiz

Campo	Tipo	Obrigatório	Descrição
<code>version</code>	inteiro (≥ 1)	sim	Versão do formato de configuração (atualmente 1).
<code>postgres</code>	objeto	não	Bloco de importação relacional.
<code>mongodb</code>	objeto	não	Bloco de importação documental.
<code>cassandra</code>	objeto	não	Bloco de importação colunar.
<code>redis</code>	objeto	não	Bloco de importação chave-valor.
<code>neo4j</code>	objeto	não	Bloco de importação em grafo.

Propriedades adicionais na raiz são **rejeitadas** (`additionalProperties: false`), o que impede erros de digitação como `postgress` passarem despercebidos.

0.4.2.2 4.2.2 Mapeamento de colunas (`columnSpec`)

Cada **folha** do mapa `columns` descreve como um valor do CSV alimenta um atributo no destino. A chave JSON no mapa `columns` identifica o campo por omissão; quando o cabeçalho CSV ou o nome no schema diferem dessa chave, usam-se os campos explícitos abaixo.

Campo	Tipo	Obrigatório	Descrição
<code>csv_column</code>	<i>string</i> ou inteiro (≥ 0)	não	Nome do cabeçalho CSV ou índice da coluna (base 0). Se omitido, a chave JSON é o cabeçalho CSV.

Campo	Tipo	Obrigatório	Descrição
<code>schema_column</code>	<i>string</i>	não	Nome do atributo no SGBD destino. Se omitido, usa-se a chave JSON.
<code>is_key</code>	booleano	não	Marca chave primária, chave de MERGE (Neo4j) ou chave Redis.
<code>db_type</code>	<i>string</i>	não	Tipo SQL/CQL para geração de DDL (PostgreSQL, Cassandra).
<code>db_column / alias_db</code>	<i>string</i>	não	Sinónimos depreciados de <code>schema_column</code> (mantidos por retrocompatibilidade).

Forma mínima — coluna CSV e destino com o mesmo nome:

```
"product_id": {}
```

Renomeação explícita — cabeçalho CSV distinto do atributo no schema:

```
"timestamp": { "schema_column": "event_time" }
```

Índice numérico — útil quando o CSV não possui cabeçalho ou o nome é ambíguo:

```
"logical_key": { "csv_column": 0, "is_key": true }
```

Chave lógica distinta do CSV e do schema:

```
"redis_key": { "csv_column": "user_id", "schema_column": "session_id",
↪  "is_key": true }
```

0.4.2.3 4.2.3 Entidade (entity)

Uma **entidade** representa um alvo lógico: tabela (PostgreSQL/Cassandra), coleção (MongoDB), rótulo de nó (Neo4j) ou entidade Redis.

Campo	Tipo	Obrigatório	Descrição
<code>columns</code>	objeto recursivo	sim	Mapa de mapeamentos; ver abaixo.
<code>filters</code>	lista	não	Predicados sobre o CSV completo antes da materialização.
<code>cassandra_partition</code>	lista de <i>strings</i>	não	Colunas CSV que formam a chave de partição.
<code>cassandra_cluster</code>	lista de <i>strings</i>	não	Colunas de agrupamento (<i>clustering</i>).
<code>nested</code>	objeto	não	Depreciado — preferir chaves aninhadas dentro de <code>columns</code> .

Aninhamento MongoDB via chaves JSON: objetos aninhados em `columns` produzem subdocumentos BSON, espelhando a estrutura de um documento JSON:

```
"columns": {
  "product_id": {},
  "category": {
    "category_id": {},
    "category_name": {}
  },
  "stock": {
    "quantity_available": {},
    "last_restock_date": {}
  }
}
```

Backends **planos** (PostgreSQL, Redis, Cassandra, Neo4j) aceitam apenas folhas em `columns`; aninhamento é rejeitado na validação cruzada.

Filtros — cada item exige `column` e `operator`:

Operador	value	Efeito
<code>==, !=, >, <, >=, <=</code>	escalar	Comparação sobre a coluna CSV.

Operador	value	Efeito
<code>in, not_in</code>	lista	Pertinência ao conjunto.
<code>each</code>	escalar (opcional)	Particiona a entidade por valores distintos da coluna; sufixo opcional em <code>target_suffix</code> .

Exemplo do e-commerce — apenas linhas de estoque alimentam o inventário:

```
"filters": [{ "column": "action", "operator": "==", "value": "stock" }]
```

0.4.2.4 4.2.4 Blocos por backend

Cada backend opcional contém `connection` (credenciais) e `entities`. PostgreSQL e Neo4j admitem ainda `relationships`.

PostgreSQL — `connection` (host, port, database, user, password), `schema` (padrão `public`), entidades planas com `db_type`, e `relationships` para chaves estrangeiras:

```
"relationships": {
  "product_category": {
    "from": "products",
    "to": "categories",
    "foreign_key": "category_id",
    "references_key": "category_id"
  }
}
```

MongoDB — `connection.uri`, `connection.database`; entidades com `columns` recursivos para subdocumentos.

Cassandra — `connection.hosts`, `keyspace`; entidades com `cassandra_partition` / `cassandra_cluster` e renomeação via `schema_column`:

```
"timestamp": { "schema_column": "event_time" },
"cassandra_partition": ["user_id"],
"cassandra_cluster": ["timestamp"]
```

Redis — exatamente uma coluna `is_key` por entidade; demais campos compõem o valor JSON.

Neo4j — nós em **entities** (um **is_key** cada); **relationships** declaram arestas tipadas com propriedades opcionais:

```
"relationships": {
  "PURCHASED": {
    "from": "User",
    "to": "Product",
    "type": "PURCHASED",
    "columns": { "order_number": { "is_key": true }, "quantity": {} }
  }
}
```

0.4.2.5 4.2.5 Validação estática e cruzada

A validação ocorre em **duas camadas**:

1. **JSON Schema** (`config_parser.validate_config`) — sintaxe, tipos e propriedades permitidas.
2. **Validação cruzada CSV** (`validation.validate_import_config`) — colunas referenciadas existem no CSV; filtros coerentes; FKs PostgreSQL e arestas Neo4j referenciam entidades válidas; backends planos não usam **columns** aninhados; índices **csv_column** dentro do intervalo.

Qualquer falha levanta `BusinessException` e **impede** o início da importação, conforme o objetivo específico do TCC.

0.4.3 4.3 Algoritmo de execução da importação

O núcleo da ferramenta está em `runner.run_import`. A seguir descreve-se o fluxo de alto nível desde a linha de comando até a persistência em cada SGBD.

0.4.3.1 4.3.1 Fases do algoritmo

1. **Entrada** — caminho do CSV, caminho do JSON de configuração e flags: `--dry-run`, `--create-schema` / `--no-create-schema`, `--only` (lista de backends).
2. **Carregar configuração** — `load_config`: parse JSON + `jsonschema.validate` contra o schema embutido.
3. **Carregar CSV** — `load_csv_with_inference`: leitura com `dtype=str` (evita erros de *parse* em timestamps com `+`) e inferência de *kind* por coluna (`integer`, `float`, `datetime`, `string`, `empty`).
4. **Validar config × CSV** — `validate_import_config`: checagens semânticas descritas na seção 4.2.5.

5. **Para cada backend** presente na configuração e não filtrado por `--only`:

- **Para cada entidade** declarada em `entities`:

- Aplicar filtros (exceto operador `each`) sobre o `DataFrame` completo.
- Expandir partições `each` (opcional) — uma entidade lógica pode gerar várias tabelas/coleções com sufixo.
- **Materializar** registros conforme o paradigma do backend:
 - * PostgreSQL / Cassandra: `flatten_entity_dataframe` — seleção, renomeação, deduplicação por `is_key`.
 - * MongoDB: `mongo_document_from_row` — documento recursivo a partir de `columns` aninhados.
 - * Redis: `redis_payload_from_row` — par (chave, valor JSON).
 - * Neo4j: propriedades de nós; depois `MERGE` de arestas a partir de `relationships`.
- Conectar ao SGBD (omitido em `--dry-run`).
- Criar schema se `--create-schema` (DDL PostgreSQL/Cassandra).
- Persistir (`INSERT`, `insert_many`, `SET`, `MERGE` Cypher, etc.).

6. **Saída** — linhas de log com contagens por entidade/backend.

0.4.3.2 4.3.2 Pseudocódigo

```
função run_import(csv_path, config_path, dry_run, create_schema, only):
    config ← load_config(config_path)
    df, kinds ← load_csv_with_inference(csv_path)
    validate_import_config(config, df, kinds)

    para cada backend em (postgres, mongodb, cassandra, redis, neo4j):
        se backend nao esta em config ou nao esta em only: continuar
        linhas ← run_{backend}_import(config[backend], df, kinds,
                                     dry_run, create_schema)

        registrar linhas no log
```

Dentro de cada `run_*_import`, para cada entidade:

```
df_filtrado ← apply_filters(df, filtros_sem_each)
para cada (nome_partição, df_parte) em expand_each(...):
    registros ← materializar(df_parte, entidade, backend)
    se não dry_run: conectar, DDL se necessário, gravar registros
```

0.4.3.3 4.3.3 Diagrama do fluxo

A Figura 4 resume o pipeline de execução descrito acima. O diagrama-fonte Mermaid encontra-se em `docs-tcc/images/figure4-import-algorithm.mmd` para futuras revisões.

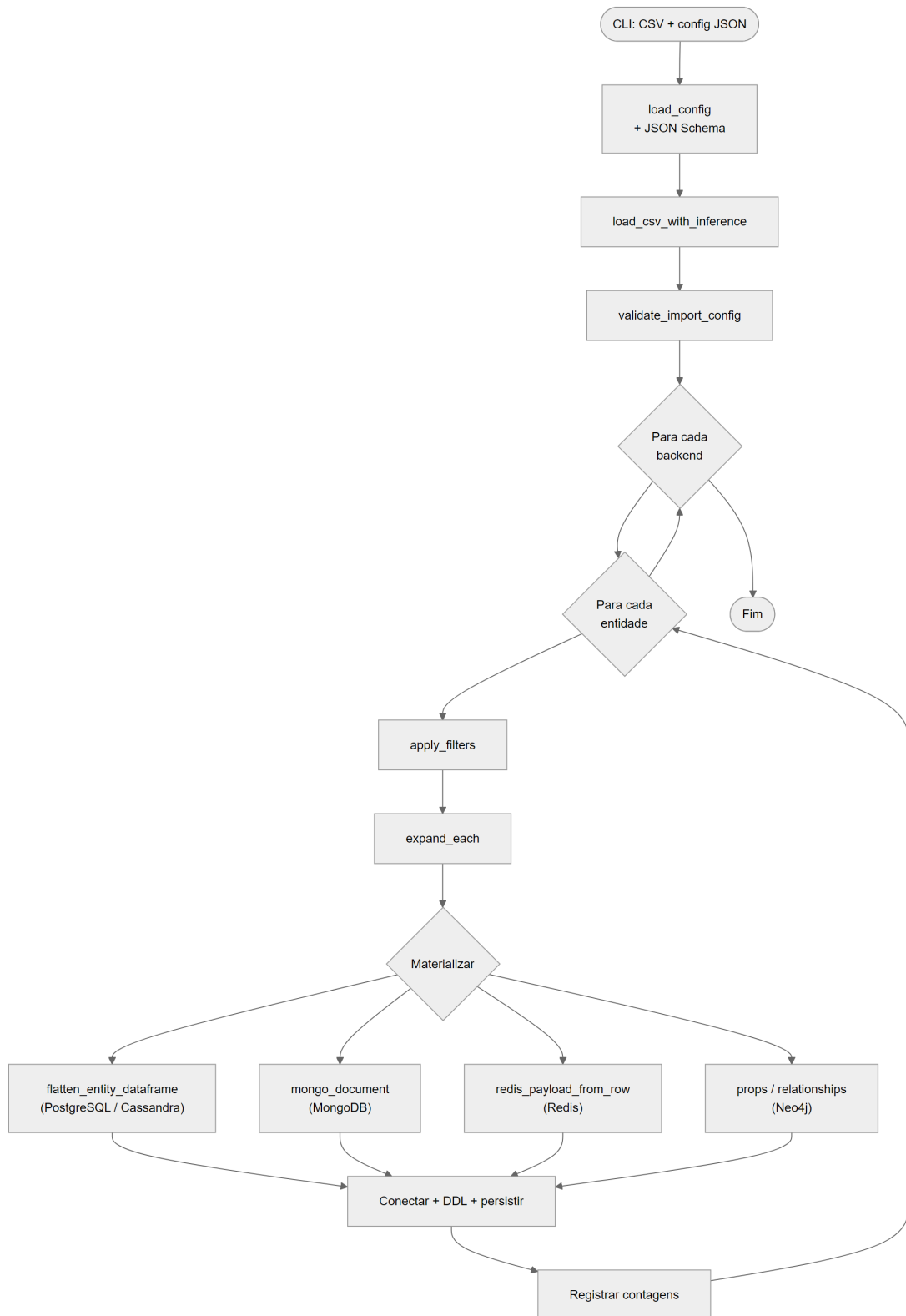


Figura 4 – Algoritmo de execução da importação Polyglot Import CSV.

Fonte: Elaborado pelo autor (2026).

0.4.3.4 4.3.4 Exemplo: uma linha `action=stock` no e-commerce

Considere uma linha do `ecommerce_join.csv` com `action = stock`, contendo `product_id`, `category_id`, `quantity_available`, `timestamp`, etc.

Backend	Entidade	Filtro aplicado	Saída materializada
PostgreSQL	<code>inventory</code>	<code>action == stock</code>	Linha plana: <code>product_id</code> , <code>quantity_available</code> , <code>last_restock_date</code> , <code>price</code> .
MongoDB	<code>product_catalog</code>	<code>action == stock</code>	Documento com campos de produto e subobjetos <code>category</code> e <code>stock</code> aninhados.
Cassandra	<code>user_activity_log</code>	<code>log</code> (todas as linhas)	Registro com <code>timestamp</code> renomeado para <code>event_time</code> , <code>action</code> para <code>event_type</code> ; PK composta (<code>user_id</code> , <code>timestamp</code>).
Redis	—	—	Entidades Redis usam outros filtros (<code>add_to_cart</code> , <code>select_product</code>); esta linha não alimenta Redis.
Neo4j	<code>Product</code>	<code>action == stock</code>	Nó <code>:Product</code> com <code>product_id</code> como chave de <code>MERGE</code> .

Assim, um **único CSV largo** alimenta destinos heterogêneos: tabelas relacionais filtradas por tipo de evento, documentos aninhados, log colunar com renomeação de colunas e nós de grafo — materializando a persistência poliglota descrita no capítulo 2.

0.4.4 4.4 Validação e execução

Erros de configuração levantam `BusinessException` antes de qualquer conexão. O modo `--dry-run` lista contagens por entidade sem contatar os SGBDs. O *driver* Apache Cassandra é importado de forma tardia para permitir `--dry-run` em ambientes onde a extensão `C` do *driver* não está disponível (por exemplo, versões recentes do Python).

Testes automatizados (`pytest`) cobrem validação de schema, mapeamento de colunas (`csv_column`, `schema_column`, aninhamento), filtros e *smoke tests* de validação + *dry-run*. O arquivo `docker-compose.yml` na raiz sobe PostgreSQL, Redis, MongoDB, Cassandra e Neo4j para testes de integração manuais; o script `run_example.ps1` orquestra a subida dos contêineres, um *dry-run* e a importação real com `--create-schema`.

0.4.4.1 4.4.1 Evidência de execução (cenário e-commerce)

O arquivo `data/ecommerce/ecommerce_join.csv` contém 32 linhas de eventos (`stock`, `purchase`, `add_to_cart`, `select_product`). Com `import_config.json` e a flag `--dry-run`, a ferramenta reporta as contagens abaixo sem abrir conexões com os SGBDs — útil para revisar o mapeamento antes da carga:

Backend	Destino	Registros (após filtros / deduplicação)
PostgreSQL	<code>categories</code>	8
PostgreSQL	<code>products</code>	8
PostgreSQL	<code>inventory</code>	8
PostgreSQL	<code>orders</code>	8
MongoDB	<code>product_catalog</code>	8 documentos
Cassandra	<code>user_activity_log</code>	32 linhas (todas as ações)
Redis	<code>shopping_cart</code>	8
Redis	<code>user_session</code>	8
Neo4j	nó <code>User</code>	8
Neo4j	nó <code>Product</code>	8
Neo4j	aresta <code>PURCHASED</code>	conforme linhas <code>purchase</code>

Com `docker compose up -d` e `run_example.ps1` (ou importação sem `--dry-run`), os mesmos volumes são persistidos nos cinco backends configurados, materializando o cenário poliglota da seção 2.1.

0.5 5 ATIVIDADES FUTURAS

- **Interface gráfica desktop** (semestre seguinte) que monte o comando CLI e visualize o JSON de configuração.
- **Suporte a TSV/Excel** e a *chunked* processing para CSV maiores que a RAM.
- **Operadores de filtro adicionais** e políticas de coerção de tipos configuráveis.
- **Testes de integração** contra `docker compose` em CI.
- **Generalização** do núcleo de importação para novos conectores (mensageria, *data lakes*, outros SGBDs).

0.6 CONSIDERAÇÕES FINAIS

Este TCC I apresentou a fundamentação teórica da persistência poliglota, o estado da arte em importação e modelagem de dados heterogêneos, e a proposta da ferramenta *Polyglot Import CSV* como utilitário de *bootstrap* declarativo a partir de CSV.

As principais contribuições desta etapa são: (i) um formato de configuração JSON validado estaticamente por JSON Schema, com mapeamento recursivo de colunas e filtros por valor; (ii) um protótipo em Python que materializa o mesmo CSV operacional em PostgreSQL, MongoDB, Cassandra, Redis e Neo4j; (iii) o modo `--dry-run` para revisão prévia de contagens; e (iv) o cenário e-commerce reproduzível com `ecommerce_join.csv`, `import_config.json` e `docker-compose.yml`.

As limitações reconhecidas incluem: testes automatizados predominantemente unitários (sem suíte de integração contínua contra Docker); processamento em memória do CSV inteiro; e dependência do *driver* Cassandra em ambientes Python recentes. Itens como interface gráfica, suporte a TSV/Excel, *chunked* processing e novos conectores permanecem no escopo do TCC II (Capítulo 5).

O trabalho de conclusão de curso em dezembro de 2026 poderá aprofundar a avaliação experimental (desempenho, consistência entre stores) e a interface de uso, consolidando a ferramenta como artefato de pesquisa e de código aberto alinhado à literatura sobre SGPD e modelagem poliglota de dados.

Referências

- BERLANGA, Adrián *et al.* A Unified Metamodel for NoSQL and Relational Databases. **arXiv preprint arXiv:2105.06494**, 2021.
- CHILLÓN, Alberto Hernández *et al.* Propagating Schema Changes to Code: An Approach Based on a Unified Data Model. *In:* 2023.
- **dbcrossbar: copy tabular data between databases, CSV files, and cloud storage.** <https://www.dbcrossbar.org>, 2024. Acesso em: 27 maio. 2026
- DUGGAN, Jennie *et al.* BigDAWG: A Polystore System for Big Data Analytics. *In:* 2017.
- FORD, Neal. **The Productive Programmer.** *[S.l.]*: O'Reilly Media, 2008.
- FOWLER, Martin. **PolyglotPersistence.** Disponível em: <https://martinfowler.com/bliki/PolyglotPersistence.html>, 2011. Acesso em: 27 maio. 2026
- GLAKE, Daniel *et al.* **Towards Polyglot Data Stores: Overview and Open Research Questions.**, 2022. Disponível em: <<https://arxiv.org/abs/2204.05779>>
- HACKOLADE. **Polyglot Data Modeling.** <https://hackolade.com/polyglot-data-modeling.html>, 2024. Acesso em: 27 maio. 2026
- HECHT, R.; JABLONSKI, S. NoSQL Evaluation: A Use Case Oriented Survey. *In:* 2011.
- KHINE, Pwint Phyu; WANG, Zhaoshun. A Review of Polyglot Persistence in the Big Data World. **Information**, v. 10, n. 4, p. 141, 2019.
- KIEHN, Felix *et al.* Polyglot Data Management: State of the Art & Open Challenges. **Proceedings of the VLDB Endowment**, v. 15, n. 12, p. 3750–3753, 2022.
- ROY-HUBARA, Naama; SHOVAL, Peretz; STURM, Arnon. Selecting Databases for Polyglot Persistence Applications. **Data & Knowledge Engineering**, v. 137, p. 101950, 2022.
- SADALAGE, Pramod J.; FOWLER, Martin. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence.** *[S.l.]*: Pearson Education, 2013.

- SILVA, Hudson Afonso Batista da; BORNIA, Luís Gustavo; MELLO, Ronaldo dos Santos. Um Estudo sobre Modelagem Poliglota de Dados. *In: Sociedade Brasileira de Computação*, 2024.
- TAN, Rui *et al.* Enabling Query Processing Across Heterogeneous Data Models: A Survey. *In: 2017*.
- WANG, L. *et al.* A Middleware for Polyglot Persistence and Data Portability of Big Data PaaS Cloud Applications. **Computers, Materials & Continua**, v. 65, n. 2, p. 1399–1416, 2020.
- YE, Feng *et al.* A Benchmark for Performance Evaluation of a Multi-Model Database vs. Polyglot Persistence. **Journal of Database Management**, v. 34, n. 3, p. 1–20, 2023.